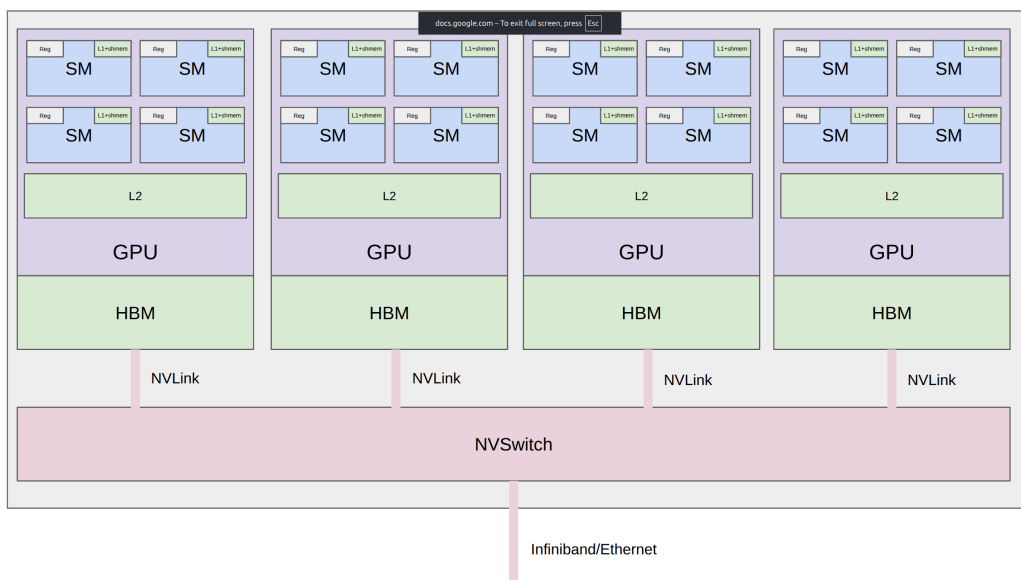


课程笔记

CS336 2026 Lecture 7: Parallelism 与分布式训练基础

基于 Stanford CS336 Spring 2026 官方讲义整理

2026 年春季



课程/频道: Stanford CS336

发布日期: Spring 2026

材料类型: 官方可执行讲义 lecture07-slides.py

课程主页: <https://cs336.stanford.edu/>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 本讲主线：把内存层次扩展到数据中心	2
1.1 为什么要多 GPU	2
1.2 泛化的层次结构	3
1.3 本讲结构	3
1.4 本章小结	3
2 Collective Operations：分布式训练的通信积木	3
2.1 Rank 与 world size	3
2.2 基础原语：broadcast、scatter、gather、reduce	4
2.3 工作马：all-gather、reduce-scatter、all-reduce	6
2.4 All-to-all：MoE 的通信核心	7
2.5 本章小结	7
3 硬件互连：拓扑决定并行策略	7
3.1 从 PCIe 到 NVLink/NVSwitch	7
3.2 RDMA、InfiniBand 与 RoCE	8
3.3 NCCL：把 collective 映射到底层路径	9
3.4 本章小结	9
4 torch.distributed：从原语到可运行代码	9
4.1 初始化分布式环境	9
4.2 All-reduce 示例	10
4.3 Reduce-scatter + all-gather 示例	10
4.4 Barrier 和同步	11
4.5 本章小结	11
5 通信 Benchmark：测的是带宽，也是算法	12
5.1 为什么要单独测 collective	12
5.2 All-reduce 的通信量直觉	12
5.3 Reduce-scatter 与 all-reduce 的关系	12
5.4 常见测量陷阱	13
5.5 本章小结	13
6 Data Parallelism：沿 batch 维切分	13
6.1 基本策略	13
6.2 梯度同步是唯一差异	14
6.3 通信账本	15
6.4 为什么引出 FSDP/ZeRO	15
6.5 本章小结	15
7 Tensor Parallelism：沿 width 维切分	16
7.1 基本策略	16
7.2 每层都要恢复完整激活	16

7.3	与生产级 Megatron 张量并行的关系	17
7.4	反向传播为什么更复杂	17
7.5	本章小结	17
8	Pipeline Parallelism: 沿 depth 维切分	18
8.1	基本策略	18
8.2	Micro-batches 与 send/recv	18
8.3	通信形态与硬件适配	19
8.4	Backward 与调度复杂度	19
8.5	本章小结	19
9	并行策略的比较与组合	20
9.1	三种基本切分维度	20
9.2	Recompute、memory、communication 三角	20
9.3	为什么 JAX/TPU 路线看起来不一样	20
9.4	本章小结	21
10	工程 Checklist: 写分布式训练代码前先问什么	21
10.1	状态在哪里	21
10.2	通信何时发生	21
10.3	硬件是否匹配策略	21
10.4	本章小结	22
11	拓展阅读	22
12	总结与延伸	22

1 本讲主线：把内存层次扩展到数据中心

Lecture 5 和 Lecture 6 讨论的是单 GPU 内部的并行：怎样让算子少碰 HBM，怎样通过 fusion、tiling、Triton kernel 把数据尽量留在寄存器、shared memory 或 cache 里。Lecture 7 把同一个问题放大到多 GPU 和多节点：当模型状态、梯度、优化器状态和激活放不下一张卡，或者训练时间要求我们使用更多 FLOPs 时，系统瓶颈从 HBM traffic 扩展为跨 GPU、跨节点通信。

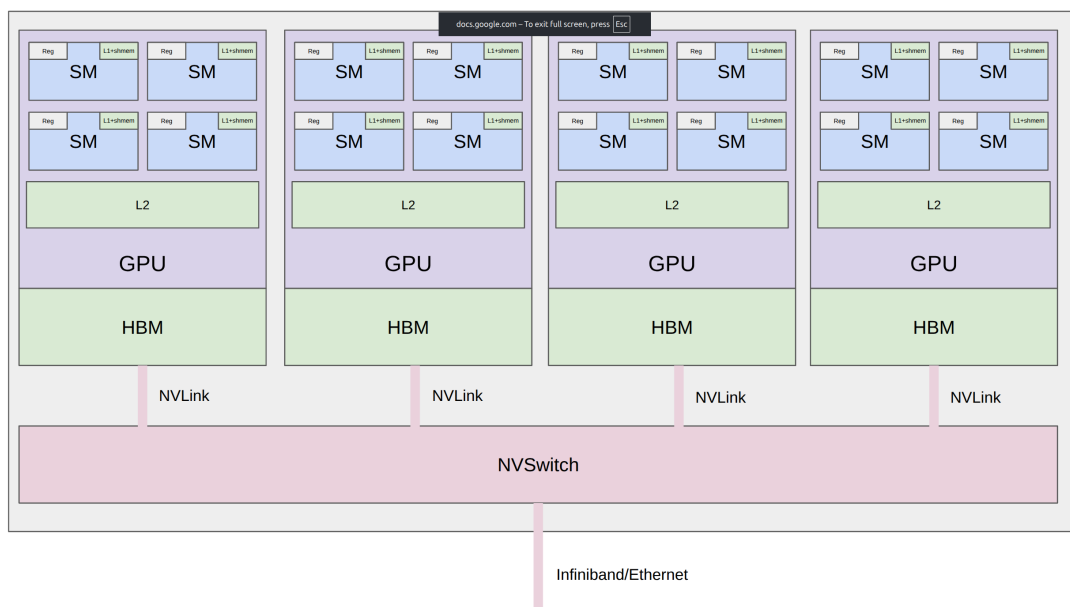


图 1: 现代 GPU 节点：多张 GPU 通过 NVLink/NVSwitch 连接，节点再通过 InfiniBand/RoCE 与更大集群相连。

来源：CS336 Spring 2026 Lecture 7 官方可执行讲义引用图。

本讲有一个非常统一的主题：**compute 离 data 总是有距离，性能工程就是组织计算来避免数据搬运成为瓶颈**。在单 GPU 内，数据搬运指 HBM 到片上；在多 GPU 内，数据搬运指 GPU 间通信；在多节点内，数据搬运进一步穿过 NIC、交换机、甚至 CPU 网络栈。

从单 GPU 到多 GPU 的同构问题

上一讲通过 fusion/tiling 减少 HBM 往返；这一讲通过 replication/sharding 减少跨 GPU 通信。两者不是两个独立主题，而是同一条 locality 原则在不同层次上的展开。

1.1 为什么要多 GPU

多 GPU 的动机可以分成两类。

1. **容量动机**：参数、梯度、优化器状态和激活放不进单卡 HBM。
2. **速度动机**：单卡算力不够，需要更多 GPU 的 FLOPs 缩短 wall-clock training time。

对大语言模型训练来说，容量动机往往先出现。以 AdamW 训练为例，单个参数不仅要存权重，还要存梯度、动量、一阶/二阶矩估计，混合精度训练还可能保留 fp32 master weights。粗略地，参数量 P 对应的训练状态可能是 $10P$ 到 $20P$ bytes 的量级。激活内存又随 batch size、sequence length、layer count 增长。

多 GPU 不是“自动变快”

多 GPU 只增加潜在 compute。能否变快，取决于每个 step 中本地计算是否足够多，通信是否足够少，通信是否能与计算重叠，以及硬件拓扑是否支持所选并行策略。

1.2 泛化的层次结构

官方讲义把层次结构按从快到慢整理如下：

层次	典型介质	优化含义
单节点、单 GPU 片上	registers、L1、shared memory	fusion、tiling、kernel design。
单节点、单 GPU 片外	HBM	避免中间张量写回，提升 arithmetic intensity。
单节点、多 GPU	NVLink、NVSwitch	适合通信频繁的 tensor parallel。
多节点、多 GPU	InfiniBand、RoCE、Ethernet	更适合粗粒度 data/pipeline parallel，必须控制通信频率。

层次越远，抽象越不能救你

PyTorch 可以隐藏 API 细节，NCCL 可以选择通信路径，但它们不能改变物理带宽和延迟。把高频小通信放到慢网络上，仍然会慢。

1.3 本讲结构

本讲分成两段：

- **分布式通信积木**：collective operations、硬件互连、NCCL、`torch.distributed`、通信 benchmark。
- **分布式训练策略**：data parallelism、tensor parallelism、pipeline parallelism。官方讲义用深 MLP 的最小实现讲清楚每种策略的切分维度。

1.4 本章小结

Lecture 7 的核心不是列举并行名词，而是让读者学会把一个训练 step 拆成“本地算多少、跨设备搬多少、何时同步、状态在哪里”。这个视角会贯穿后续 FSDP/ZeRO、expert parallel、sequence parallel 和 3D/4D parallelism。

2 Collective Operations：分布式训练的通信积木

2.1 Rank 与 world size

分布式程序中，每个参与者叫一个 rank；参与通信的总设备数叫 world size。比如四张 GPU 的进程组中，rank 可以是 0、1、2、3，world size 是 4。Collective operation 不是 rank 与 rank 之间任意发消息，而是所有 rank 共同参与的标准通信协议。



图 2: rank/world size 的最小例子：每个设备有一个 rank，所有 rank 构成一个通信世界。

来源：CS336 Spring 2026 Lecture 7 官方可执行讲义配图。

Collective 的含义

Collective 表示“所有参与者一起执行一个通信模式”。如果某个 rank 没有进入同一个 collective，其他 rank 会等待甚至死锁。因此分布式代码中，控制流一致性和 shape 一致性与数学正确性同样重要。

2.2 基础原语：broadcast、scatter、gather、reduce

四个基础原语帮助我们建立语言：

- **Broadcast**: rank 0 的完整张量复制到所有 rank。常用于 rank 0 读取 checkpoint 后同步给其他 rank。
- **Scatter**: rank 0 的张量切成多份，分别发给各 rank。它是理解 reduce-scatter 的前置概念。
- **Gather**: 各 rank 的分片收集到 rank 0。它是 scatter 的反向。
- **Reduce**: 各 rank 的张量按 sum/min/max 等操作归约到 rank 0。

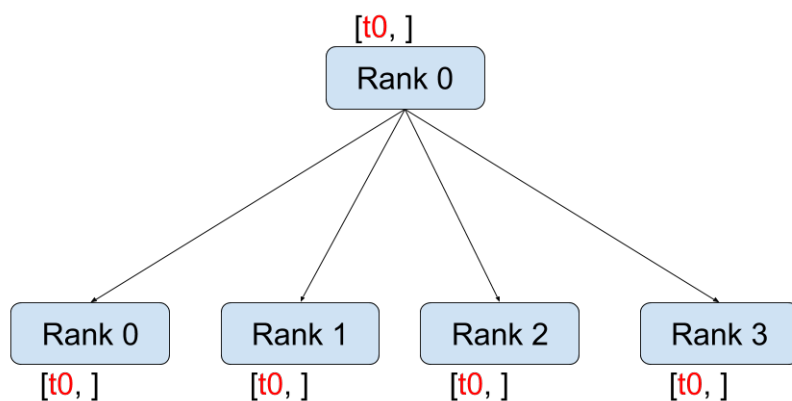


图 3: Broadcast：一个 rank 的数据复制给所有 rank。

来源：PyTorch distributed tutorial collective diagram。

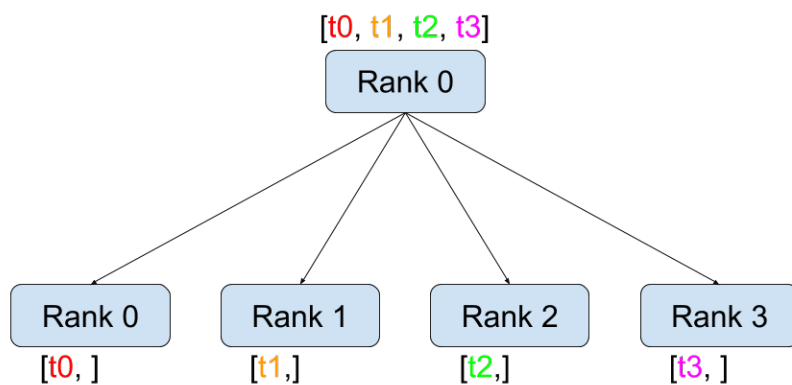


图 4: Scatter: 源 rank 把张量切片分发给不同 rank。

来源: *PyTorch distributed tutorial collective diagram.*

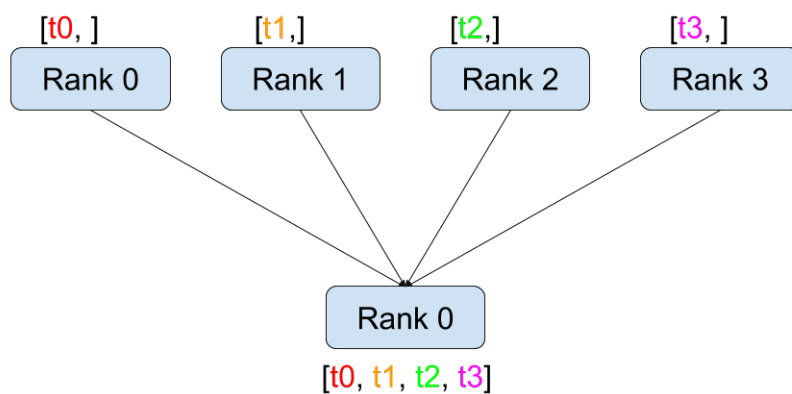


图 5: Gather: 多个 rank 的分片汇总到一个 rank。

来源: *PyTorch distributed tutorial collective diagram.*

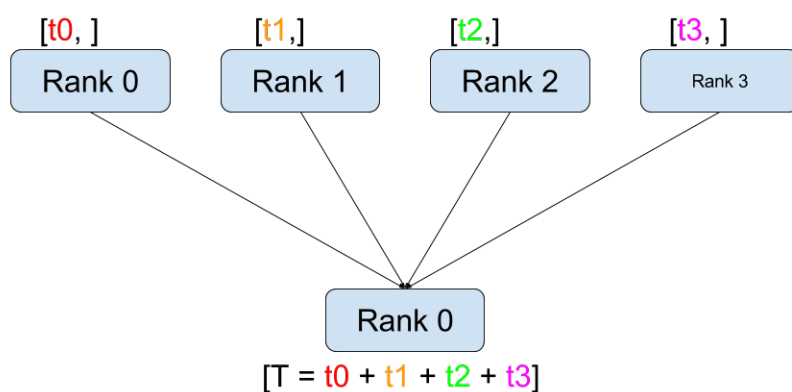


图 6: Reduce: 多个 rank 的数据被求和或按其他可结合操作归约到一个 rank。

来源: *PyTorch distributed tutorial collective diagram.*

Reduce 要求什么

Reduce 通常要求操作可结合、可交换，例如 sum、min、max。训练里最常见的是梯度求和或求平均。若通信顺序不同，浮点加法会有细微数值差异，这也是分布式训练可复现性更难的原因之一。

2.3 工作马：all-gather、reduce-scatter、all-reduce

真正支撑大模型训练的通常是三类 workhorse collectives。

All-gather：每个 rank 起初只有一个分片，结束后每个 rank 都拥有完整拼接结果。在 FSDP 和 ZeRO-3 中，每个 rank 只常驻一部分参数；前向计算某一层前，再通过 all-gather 临时得到完整参数。

Reduce-scatter：每个 rank 起初有一份完整张量，先跨 rank reduce，再把结果按 shard 分给不同 rank。数据并行反向传播后，如果不想让每个 rank 都保存完整梯度，可以用 reduce-scatter 得到梯度分片。

All-reduce：每个 rank 起初有一份张量，结束后每个 rank 都得到跨 rank reduce 的完整结果。DDP 梯度同步就是典型 all-reduce。

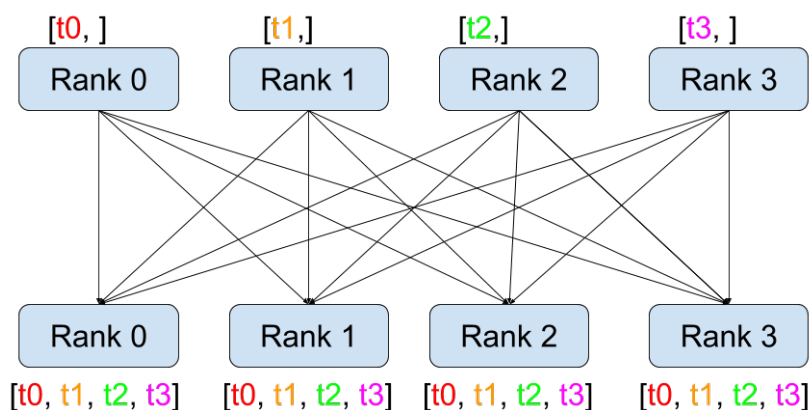


图 7: All-gather：每个 rank 都收集到所有分片。

来源：PyTorch distributed tutorial collective diagram。

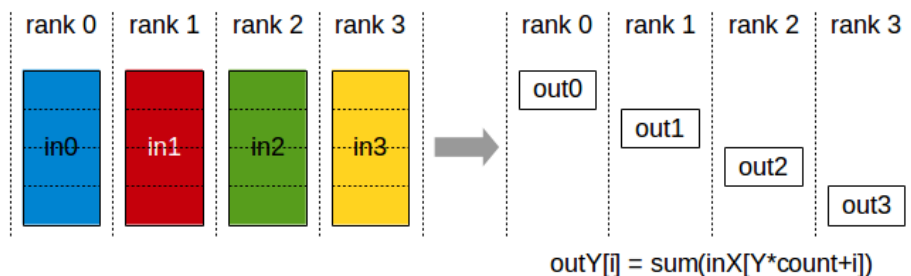


图 8: Reduce-scatter：先归约，再把归约结果分片分给各 rank。

来源：NVIDIA NCCL user guide collective diagram。

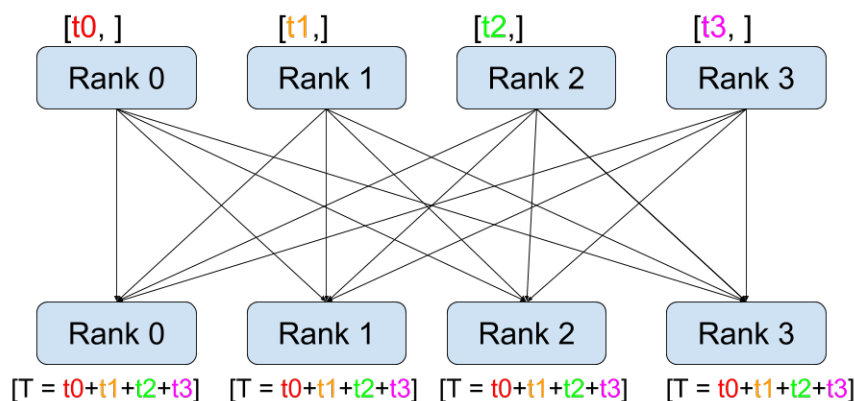


图 9: All-reduce: 所有 rank 最终获得同一份归约结果。

来源: *PyTorch distributed tutorial collective diagram*。

最重要的恒等式

$$\text{all-reduce} = \text{reduce-scatter} + \text{all-gather}.$$

这条恒等式不只是通信课小技巧。它解释了为什么 ZeRO/FSDP 可以把“完整梯度同步”改写成“梯度分片 + 参数临时聚合”，从而用类似通信量换取更低显存占用。

2.4 All-to-all: MoE 的通信核心

All-to-all 是更一般的模式：每个 rank 都向每个其他 rank 发送一块数据。对 MoE 来说，tokens 先按 router 分配到专家，而专家又分布在不同 rank 上，所以需要把每个 rank 上的 tokens 重新路由到拥有对应 experts 的 rank。平衡情况下，all-to-all 类似矩阵转置；不平衡情况下，它会暴露 load imbalance。

All-to-all 对负载均衡很敏感

MoE router 如果把大量 tokens 分给少数 expert/rank，不只是计算不均衡，通信也会不均衡。系统层面的 expert parallel 不能只看平均 tokens per expert，还要看最坏 rank 的负载。

2.5 本章小结

Collectives 是理解分布式训练的词汇表。Data parallel 的关键词是 all-reduce；FSDP/ZeRO 的关键词是 all-gather 和 reduce-scatter；tensor parallel 常用 all-gather 或 all-reduce；MoE/expert parallel 常用 all-to-all。后续每种并行策略，都可以还原成这些原语的组合。

3 硬件互连：拓扑决定并行策略

3.1 从 PCIe 到 NVLink/NVSwitch

家用或普通工作站中，多张 GPU 可能主要通过 PCIe 与 CPU/root complex 相连。PCIe 带宽持续提升，但它不是为 GPU 间高频大规模 collective 最优设计的。数据中心训练节点通常使用 NVLink/NVSwitch，让 GPU 之间以更高带宽、更低延迟互联。

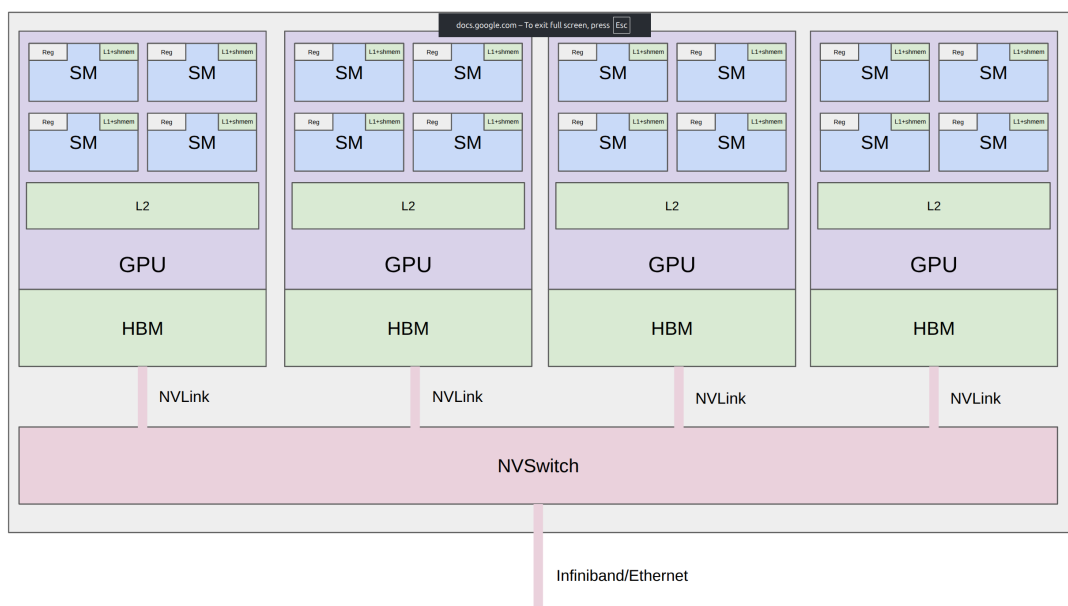


图 10: 节点内 NVLink/NVSwitch 与节点间网络共同组成分布式训练的通信层次。

来源: CS336 Spring 2026 Lecture 7 官方可执行讲义引用图。

官方讲义给出的现代典型设置是:

- 每个 node 通常有 8 张 GPU, 经由 NVLink 连接到 NVSwitch;
- B200 NVLink 5.0 约 1.8 TB/s, 而 B200 HBM 约 8 TB/s;
- 一个 pod 中可能有 256 个 nodes, 经由 InfiniBand 或类似网络连接;
- 节点间带宽量级可以低到约 0.05 TB/s, 远低于节点内 GPU 互联。

带宽比绝对数字更重要

即使 NVLink 很快, 它仍然比 HBM 慢; 即使 InfiniBand 很快, 它仍然比 NVLink 慢很多。并行策略要尊重这些比值: 高频通信放在快层次, 低频粗粒度通信才放到慢层次。

3.2 RDMA、InfiniBand 与 RoCE

普通 Ethernet 通信经常经过 CPU 和 OS 网络栈: 数据从 GPU/用户态搬到 CPU memory, 再进入 kernel socket buffer, 打包 TCP, 再交给 NIC。这个路径延迟和 CPU overhead 都高。RDMA (Remote Direct Memory Access) 的核心价值是允许一台机器直接读写另一台机器的内存, 绕过 CPU 的大量参与。InfiniBand 天然支持 RDMA; RoCE 则是在 Ethernet 上提供类似能力, 成本更低但工程上更微妙。

绕过 CPU 是训练集群的刚需

跨节点通信若每次都让 CPU 复制和打包数据, GPU 很容易等待网络。RDMA/GPUDirect RDMA 的目标, 就是让通信路径更接近“GPU memory 到 GPU memory”的直接搬运。

3.3 NCCL: 把 collective 映射到底层路径

NCCL (NVIDIA Collective Communication Library) 的作用, 是把 all-reduce、reduce-scatter、all-gather 等 collective 操作翻译成底层 GPU 间通信。它会检测拓扑, 例如 GPU 数量、NVLink、PCIe、switch、NIC, 然后选择 ring、tree 或其他算法组合, 并 launch GPU kernels 负责 send/receive。

Listing 1: PyTorch 中通常通过 NCCL backend 使用 GPU collective

```
1 if torch.cuda.is_available():
2     dist.init_process_group(
3         backend="nccl",
4         rank=rank,
5         world_size=world_size,
6     )
7 else:
8     dist.init_process_group(
9         backend="gloo",
10        rank=rank,
11        world_size=world_size,
12    )
```

NCCL 优化路径, 但不改变通信量

NCCL 可以帮你选择更好的 route 和 algorithm, 但如果算法每层都要跨节点 all-reduce 大激活, 通信量仍然巨大。并行策略先决定“通信什么、通信多频繁”, NCCL 再优化“怎么通信”。

3.4 本章小结

硬件层次直接约束并行策略: tensor parallel 通信频繁, 通常要求节点内 NVLink/NVSwitch; pipeline parallel 通信较粗粒度, 可以跨更慢链路; data parallel/FSDP 是否高效, 取决于 batch size、参数大小、通信重叠和网络带宽。

4 torch.distributed: 从原语到可运行代码

4.1 初始化分布式环境

官方讲义的最小分布式程序为每个 rank 启动一个进程, 并通过 MASTER_ADDR 和 MASTER_PORT 建立 rendezvous。真实训练中通常由 torchrun、SLURM 或集群调度器设置这些环境变量。

Listing 2: 分布式环境初始化的最小版本

```

1 def setup(rank: int, world_size: int):
2     os.environ["MASTER_ADDR"] = "localhost"
3     os.environ["MASTER_PORT"] = "15623"
4
5     if torch.cuda.is_available():
6         dist.init_process_group("nccl", rank=rank,
7                                 world_size=world_size)
8     else:
9         dist.init_process_group("gloo", rank=rank,
10                                world_size=world_size)

```

一个 rank 通常对应一个进程

单机多卡训练中, 常见模式是 one process per GPU。这样每个进程设置自己的 current device, 并参与同一个 process group。不要把“rank”简单理解成 Python 线程, 它通常是独立进程。

4.2 All-reduce 示例

Listing 3: all-reduce 会原地修改每个 rank 的 tensor

```

1 data = torch.tensor([0., 1, 2, 3], device=device) + rank
2
3 print(f"Rank {rank} before: {data}")
4 dist.all_reduce(
5     tensor=data,
6     op=dist.ReduceOp.SUM,
7     async_op=False,
8 )
9 print(f"Rank {rank} after: {data}")

```

如果 world size 为 4, rank 0 的输入是 [0, 1, 2, 3], rank 1 是 [1, 2, 3, 4], rank 2 是 [2, 3, 4, 5], rank 3 是 [3, 4, 5, 6], all-reduce sum 后每个 rank 都得到 [6, 10, 14, 18]。这正是 DDP 梯度同步的形状: 每个 rank 本地算出梯度, 然后所有 rank 拿到平均后的完整梯度。

4.3 Reduce-scatter + all-gather 示例

官方讲义紧接着执行 reduce-scatter, 再把 reduce-scatter 的输出 all-gather 回来, 用代码验证 all-reduce 的分解。

Listing 4: reduce-scatter 后每个 rank 只保留归约结果的一片

```
1 input = torch.arange(
2     world_size,
3     dtype=torch.float32,
4     device=device,
5 ) + rank
6 output = torch.empty(1, device=device)
7
8 dist.reduce_scatter_tensor(
9     output=output,
10    input=input,
11    op=dist.ReduceOp.SUM,
12    async_op=False,
13 )
14
15 gathered = torch.empty(world_size, device=device)
16 dist.all_gather_into_tensor(
17     output_tensor=gathered,
18     input_tensor=output,
19     async_op=False,
20 )
```

这段代码为什么重要

它不是为了证明一个数学恒等式而已，而是在提前铺垫 FSDP/ZeRO：如果我们愿意让每个 rank 只持有一部分状态，就可以用 reduce-scatter/all-gather 替代“所有 rank 永久持有完整状态”。

4.4 Barrier 和同步

`dist.barrier()` 会等待所有 rank 到达同一点。讲义中用 barrier 让打印输出更可读，也在 benchmark 中让计时区间一致。CUDA kernel 本身又是异步执行的，所以 GPU benchmark 还需要 `torch.cuda.synchronize()`。分布式计时里，这两个同步经常都要考虑。

分布式 deadlock 通常来自不一致

所有 rank 必须以相同顺序进入 collective，tensor shape/dtype 也必须匹配。某个 rank 多执行一次 all-reduce，或者某个 rank 因异常提前退出，都可能让其他 rank 卡住。

4.5 本章小结

`torch.distributed` 把 collective 暴露成 Python API，但真实执行由 backend 决定。GPU 上常用 NCCL，CPU fallback 可用 Gloo。写分布式代码时，要同时考虑 process group 初始化、rank/device 映射、collective 顺序、barrier、CUDA 同步和清理。

5 通信 Benchmark: 测的是带宽, 也是算法

5.1 为什么要单独测 collective

分布式训练 step time 经常由 compute 和 communication 共同决定。若只测完整训练 step, 很难知道慢在 matmul、attention、gradient all-reduce、parameter all-gather 还是 pipeline send/recv。单独 benchmark collective 可以建立硬件和软件栈的基线: 当前节点内 all-reduce 到底能跑到多少 GB/s, reduce-scatter 是否和 all-reduce 的带宽接近, 跨节点通信是否比预期慢。

Listing 5: collective benchmark 的基本结构

```
1 data = torch.randn(num_elements, device=device)
2
3 dist.all_reduce(data, op=dist.ReduceOp.SUM)
4 torch.cuda.synchronize()
5 dist.barrier()
6
7 start_time = time.time()
8 dist.all_reduce(data, op=dist.ReduceOp.SUM)
9 torch.cuda.synchronize()
10 dist.barrier()
11 duration = time.time() - start_time
```

5.2 All-reduce 的通信量直觉

在 ring all-reduce 的经典分析中, all-reduce 可以看成 reduce-scatter + all-gather。每个 rank 发送和接收大约 $2^{\frac{p-1}{p}}S$ bytes, 其中 p 是 world size, S 是 tensor size。不同工具对 bandwidth 的报告可能区分 algorithm bandwidth 和 bus bandwidth, 因此读 benchmark 时要确认公式。

官方讲义中的简化代码把发送量写成:

$$\text{sent_bytes} = S \cdot 2(p - 1),$$

再结合 rank 数和 duration 得到一个有效带宽估计。它的教学目的不是替代 NCCL tests, 而是让学生看到“通信时间”和“移动了多少数据”之间的关系。

不要只看 duration

同样是 20 ms, 传 1 GB 和传 10 GB 含义完全不同。collective benchmark 应同时报告 tensor size、world size、duration 和 effective bandwidth。

5.3 Reduce-scatter 与 all-reduce 的关系

Reduce-scatter 移动的是 all-reduce 的一半阶段; all-gather 是另一半。因此在同一网络上, 两者的有效带宽通常与 all-reduce 可比。FSDP 和 ZeRO 的吸引力就在这里: 把“完整同步后每卡保存完整状态”改写成“只保存必要分片”, 通信账本仍可由这些 collective 估算。

benchmark 是并行策略选择的输入

如果节点内 NVLink all-reduce 很快, tensor parallel 的通信压力可以被接受; 如果跨节点 bandwidth/latency 差, 跨节点 tensor parallel 会很吃亏, 此时更倾向 data parallel、pipeline parallel 或更大粒度的 sharding。

5.4 常见测量陷阱

陷阱	后果	处理方式
没有 warmup	首次初始化/建连污染结果	先跑一次 collective 再计时。
没有 CUDA synchronize	只测到 CPU launch 或排队时间	计时结束前同步 GPU。
没有 barrier	不同 rank 计时区间错位	计时前后 barrier 对齐。
tensor 太小	latency 主导, 带宽数字无意义	用多个 size 扫描 latency-bound 到 bandwidth-bound 区间。
只测单节点	误判跨节点训练表现	分别测 intra-node 和 inter-node。

5.5 本章小结

通信 benchmark 把硬件互连从“规格表”变成“训练可用带宽”。只有知道 collective 的真实成本, 才能判断一种并行策略是否值得使用, 以及哪些通信应该被 overlap、shard 或避免。

6 Data Parallelism: 沿 batch 维切分

6.1 基本策略

数据并行是最直观的策略: 每个 rank 拥有完整模型参数, 输入 batch 沿 batch dimension 切成 p 份, 每个 rank 处理本地 mini-batch, 反向传播后对梯度 all-reduce, 最后每个 rank 用同样的梯度更新同样的参数。

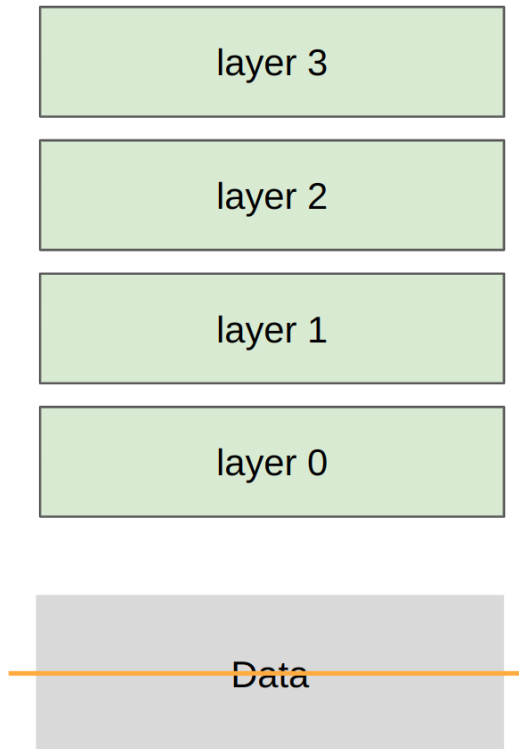


图 11: Data parallelism: 切数据, 复制模型, 通过梯度 all-reduce 保持参数一致。

来源: CS336 Spring 2026 Lecture 7 官方可执行讲义配图。

Listing 6: 数据并行: 每个 rank 取 batch 的一个切片

```
1 batch_size = data.size(0)
2 local_batch_size = batch_size // world_size
3 start_index = rank * local_batch_size
4 end_index = start_index + local_batch_size
5 data = data[start_index:end_index].to(device)
```

6.2 梯度同步是唯一差异

官方讲义刻意用 bare-bones MLP 展示: 除了每个 rank 只看本地数据, 标准训练 loop 几乎不变。关键差异是在 `loss.backward()` 后, 对每个参数的梯度执行 all-reduce。

Listing 7: DDP 的核心：反向传播后同步梯度

```

1 loss.backward()
2
3 for param in params:
4     dist.all_reduce(
5         tensor=param.grad,
6         op=dist.ReduceOp.AVG,
7         async_op=False,
8     )
9
10 optimizer.step()

```

如果各 rank 使用平均梯度并从相同参数出发，那么参数会保持一致。各 rank 的本地 loss 可以不同，因为它们看的是不同数据；但 all-reduce 后的梯度相同，更新也相同。

Data parallel 的优点和代价

它实现简单、适合慢速互连、扩展吞吐直接；但每个 rank 仍保存完整参数、梯度和优化器状态。它解决了计算并行问题，不自动解决模型状态显存问题。

6.3 通信账本

若模型参数大小为 P bytes，每个 step 需要对梯度做 all-reduce，通信量粗略是 $O(P)$ ，更细地看 ring all-reduce 每 rank 约 $2^{\frac{p-1}{p}}P$ bytes。数据并行的 compute 随本地 batch 增长，通信随参数量增长。大 batch 下 compute/communication ratio 更好，小 batch 下 all-reduce 容易主导。

为什么大 batch 对 data parallel 友好

参数梯度通信量基本不随 batch size 增长，但本地 forward/backward compute 随 batch size 增长。因此 batch 越大，越容易把通信摊薄。不过 batch 过大又会带来优化稳定性、泛化和 learning-rate scaling 问题。

6.4 为什么引出 FSDP/ZeRO

讲义在 data parallelism 末尾提示下一讲的 FSDP/ZeRO：如果每个 rank 都复制完整参数、梯度和优化器状态，显存无法随 GPU 数线性扩展。FSDP/ZeRO 的思路是把这些状态分片保存，用 all-gather 临时重建参数，用 reduce-scatter 分发梯度结果。

DDP 不等于显存线性扩展

增加 data parallel ranks 可以增加总 batch 和总吞吐，但单卡仍需容纳完整模型状态和本地激活。模型本身放不进单卡时，纯 DDP 无能为力。

6.5 本章小结

Data parallelism 沿 batch 维切分，模型复制，梯度 all-reduce。它是最基础、最稳健的并行策略，但显存复制限制明显。理解它是理解 DDP、ZeRO 和 FSDP 的起点。

7 Tensor Parallelism: 沿 width 维切分

7.1 基本策略

Tensor parallelism 把单层内部的参数矩阵沿 hidden/width 维切开, 每个 rank 只保存和计算一部分层宽。它解决的是“单层太宽、单卡放不下或算不过来”的问题。官方讲义的简化 MLP 中, 每个 rank 拥有每层参数的 $1/p$ 列, 先计算本地激活, 再 all-gather 拼回完整 hidden dimension, 供下一层使用。

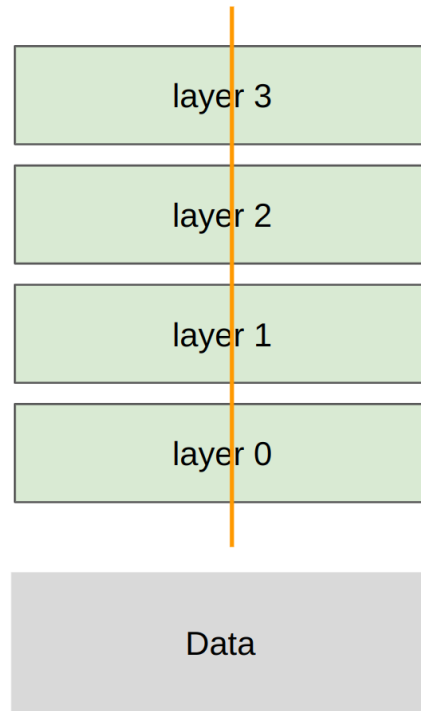


图 12: Tensor parallelism: 切每层宽度, 每层之间需要通信激活。

来源: CS336 Spring 2026 Lecture 7 官方可执行讲义配图。

Listing 8: Tensor parallel: 每个 rank 保存每层参数的一部分列

```
1 data = data.to(device)
2 batch_size = data.size(0)
3 num_dim = data.size(1)
4 local_num_dim = num_dim // world_size
5
6 params = [
7     get_init_params(num_dim, local_num_dim, rank)
8     for layer in range(num_layers)
9 ]
```

7.2 每层都要恢复完整激活

在官方简化实现中, 每层本地矩阵乘得到形状为 `batch_size x local_num_dim` 的局部激活。下一层仍需要完整 `num_dim` 输入, 因此要 all-gather 所有 rank 的局部激活并拼接。

Listing 9: Tensor parallel forward: 局部 matmul 后 all-gather 激活

```
1 for layer in range(num_layers):
2     x = x @ params[layer]
3     x = F.gelu(x)
4
5     activations = [
6         torch.empty(batch_size, local_num_dim, device=device)
7         for _ in range(world_size)
8     ]
9     dist.all_gather(tensor_list=activations,
10                    tensor=x,
11                    async_op=False)
12     x = torch.cat(activations, dim=1)
```

Tensor parallel 的通信频率很高

数据并行通常每 step 对梯度通信；tensor parallel 往往每一层 forward/backward 都要通信激活或梯度。因此它强烈依赖高速互连，实践中常限制在同一节点的 NVLink/NVSwitch 域内。

7.3 与生产级 Megatron 张量并行的关系

教学代码用 all-gather 拼激活来解释最基本思路。生产级 Transformer tensor parallel 更精细：MLP 的第一层可以 column parallel，第二层 row parallel；attention 的 QKV/head 可以按 head 或 hidden 维切分；一些同步点用 all-reduce，一些用 reduce-scatter/all-gather。目标是减少每层同步次数和通信量，同时保持矩阵乘法形状对 Tensor Cores 友好。

切 width 的本质

Tensor parallel 不是“把模型任意切开”，而是利用线性层和 attention head 的代数结构，让每个 rank 做一部分矩阵乘法。切分维度必须与算子结构兼容，否则会产生过多通信。

7.4 反向传播为什么更复杂

官方讲义把 tensor parallel 的 backward 留作作业。这不是省略小细节，而是因为 backward 中激活梯度、权重梯度和输入梯度的切分关系不同。某些方向需要 all-reduce 求和，某些方向需要 all-gather 或 reduce-scatter。训练系统框架的复杂度，很大一部分来自把 forward/backward 的 sharding 语义都维护正确。

Tensor parallel 的正确性边界

只写 forward demo 不代表训练可用。完整 tensor parallel 需要处理 backward、optimizer state、randomness/dropout、layernorm、attention mask、checkpointing 和 mixed precision。

7.5 本章小结

Tensor parallelism 沿层宽切分参数和计算，能降低单卡参数/计算负担，但每层通信频繁，因此通常需要节点内高速互连。它是大模型训练中最重要的“节点内模型并行”策略之一。

8 Pipeline Parallelism: 沿 depth 维切分

8.1 基本策略

Pipeline parallelism 把模型按层深度切成多个 stages，每个 rank 拥有连续的一段 layers。数据从第一个 stage 进入，经过 rank 0 的 layers 后发送给 rank 1，继续向后传。它沿 depth 维切分模型，因此可以把很深的网络放到多张 GPU 上。

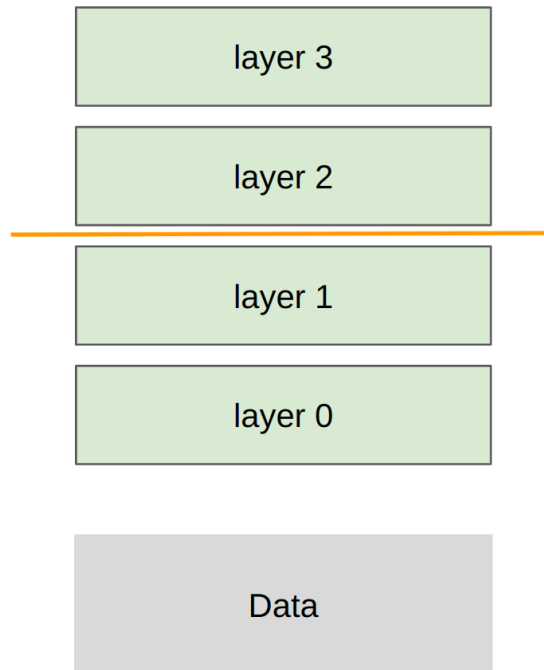


图 13: Pipeline parallelism: 每个 rank 拥有一段层，激活在 stage 间传递。

来源: CS336 Spring 2026 Lecture 7 官方可执行讲义配图。

Listing 10: Pipeline parallel: 每个 rank 只初始化自己的层

```
1 local_num_layers = num_layers // world_size
2 local_params = [
3     get_init_params(num_dim, num_dim, rank)
4     for layer in range(local_num_layers)
5 ]
```

8.2 Micro-batches 与 send/recv

如果一次把整个 batch 送进 pipeline，后面的 rank 在开始时会空等，前面的 rank 在结束后会空等。解决方法是把 batch 切成 micro-batches，让不同 micro-batches 同时处在不同 pipeline stages 中。官方代码展示了最小 forward pipeline：rank 0 拆分数据，其他 rank 预分配 activation buffer；每个 stage 收前一 rank 的 activation，计算本地 layers，再发给下一 rank。

Listing 11: Pipeline forward: 接收、计算、发送

```

1 for x in micro_batches:
2     if rank - 1 >= 0:
3         dist.recv(tensor=x, src=rank - 1)
4
5     for param in local_params:
6         x = x @ param
7         x = F.gelu(x)
8
9     if rank + 1 < world_size:
10        dist.send(tensor=x, dst=rank + 1)

```

Pipeline 的核心瓶颈是 bubble

Pipeline parallel 减少了单卡模型状态，但会产生 pipeline bubble: 某些时刻部分 stages 没有 micro-batch 可算。micro-batch 越多，bubble 相对越小，但 activation 管理和调度更复杂。

8.3 通信形态与硬件适配

Pipeline parallel 的通信通常是相邻 stages 之间传 activation，频率按 micro-batch 和 layer partition 决定。相比 tensor parallel 每层 collective，pipeline 通信更粗粒度，更容易跨节点使用。但如果 stage 划分不均衡，最慢 stage 会决定全局吞吐；如果 micro-batch 太少，bubble 会吞掉很多 GPU 时间。

Pipeline 适合慢链路的原因

Pipeline 不要求每层都做全体 all-reduce，而是主要在 stage 边界传 activation。通信模式更局部、更粗粒度，因此比 tensor parallel 更能容忍较慢的跨节点网络。

8.4 Backward 与调度复杂度

完整 pipeline training 不只是 forward send/recv。Backward 需要反向传递 activation gradients，还要处理 weight update 的时序。实际系统有 GPipe、PipeDream、1F1B、interleaved 1F1B、zero-bubble 等调度策略，它们在 memory、bubble、staleness 和实现复杂度之间取舍。Lecture 7 只展示 minimal forward，是为了先建立 depth sharding 的基本形态。

Pipeline 并不自动负载均衡

按层数均分不一定按时间均分。attention、MLP、embedding、输出头、MoE layers 的计算成本不同，stage 划分要按 profile 和 memory 共同决定。

8.5 本章小结

Pipeline parallelism 沿 depth 切分模型，降低单卡模型状态压力，通信比 tensor parallel 更粗粒度，但需要 micro-batching 减少 bubble，并在完整训练中处理复杂 backward 调度。

9 并行策略的比较与组合

9.1 三种基本切分维度

Lecture 7 的三个实现可以用一句话概括：

- **Data parallelism**: 切 batch, 复制模型, 通信梯度。
- **Tensor parallelism**: 切 width, 分片层内参数/激活, 通信频繁。
- **Pipeline parallelism**: 切 depth, 分片 layers, 通信 stage boundary activations。

策略	切分维度	主要通信	硬件偏好
Data parallel	batch	梯度 all-reduce	慢链路也可, batch 大更友好。
FSDP/ZeRO	model states	参数 all-gather、梯度 reduce-scatter	中高速网络, 依赖 overlap。
Tensor parallel	hidden/width/head	每层 all-gather/all-reduce	节点内 NVLink/N-VSwitch。
Pipeline parallel	layers/depth	stage 间 send/recv	可跨节点, 但要控制 bubble。
Expert parallel	experts	all-to-all token routing	依赖负载均衡和高速网络。
Sequence parallel	sequence length	激活与归一化分片	常与 tensor parallel 组合。

选择并行策略的第一原则

先让高频通信留在高速层次, 再把低频粗粒度通信放到慢速层次。通常: tensor parallel 放节点内, data/FSDP/pipeline 扩展到节点间, expert parallel 要特别关注 all-to-all 负载均衡。

9.2 Recompute、memory、communication 三角

讲义总结中提到一个关键三角: 可以选择重计算、存内存、或者存在另一张 GPU 的内存里并通信。比如 activation checkpointing 用更多 compute 换更少 memory; FSDP 用 communication 换更少 replicated state; pipeline 把 layers 存在不同 GPU, 用 activation communication 换单卡容量。

没有免费午餐, 只有换瓶颈

显存不够时, 你可以重算、分片、通信、offload。每种选择都把压力挪到 compute、network、CPU memory 或调度复杂度上。系统设计的关键是让最便宜的资源承担更多压力。

9.3 为什么 JAX/TPU 路线看起来不一样

官方讲义简短提到 JAX/TPU: 定义模型、sharding strategy, 让编译器处理更多细节。TPU pods 的拓扑和 XLA/SPMD 编译栈让用户以更高层方式表达 sharding。PyTorch 路线在这讲中显得更“手工”, 但教育价值很高: 它迫使我们看见底层 primitives 如何组合成训练系统。

抽象层越高，越不能忘记通信账本

无论使用 JAX 编译器、PyTorch FSDP，还是手写 collectives，底层都绕不开数据移动。高层 API 可以减少出错面，但不会取消硬件层次。

9.4 本章小结

Data/tensor/pipeline parallelism 不是互斥选项，而是沿 batch、width、depth 三个不同维度切分训练问题。真实大模型通常组合使用：节点内 tensor parallel，节点间 data/FSDP/pipeline，MoE 再叠 expert parallel。组合的核心仍然是通信频率和硬件层次匹配。

10 工程 Checklist：写分布式训练代码前先问什么

10.1 状态在哪里

首先要明确参数、梯度、优化器状态、激活分别在哪里：

- 参数是复制还是分片？
- 梯度是完整 all-reduce 后复制，还是 reduce-scatter 后分片？
- 优化器状态每卡完整保存，还是按参数 shard 保存？
- 激活在本地保存、checkpoint 重算，还是跨 stage 发送？

10.2 通信何时发生

其次要标出每个训练 step 的同步点：

- forward 前是否 all-gather 参数？
- 每层 forward/backward 是否有 tensor-parallel collective？
- backward 后是否 all-reduce/reduce-scatter 梯度？
- pipeline stage 间 send/recv 是否能与计算重叠？

10.3 硬件是否匹配策略

场景	风险信号	常见调整
Tensor parallel 跨节点	每层通信穿过慢网络	限制 TP 到节点内，跨节点用 DP/FSDP/PP。
Data parallel batch 太小	梯度 all-reduce 占比高	增大 batch/gradient accumulation，或 overlap 通信。
Pipeline micro-batch 太少	bubble 大、GPU 空闲	增加 micro-batches 或调整 schedule。

FSDP all-gather 频繁	参数通信阻塞 forward	prefetch、overlap、合理 wrap policy。
MoE all-to-all 不均衡	少数 rank 拖慢 step	router load balancing、capacity factor、expert placement。

最终的判断标准

并行策略不是按名字先进程度排序，而是按 end-to-end step time、memory headroom、稳定性和可维护性共同评估。能用简单 data parallel 解决的问题，不必过早引入复杂 pipeline；单卡放不下时，再逐步引入 state sharding 和 model parallel。

10.4 本章小结

分布式训练工程的第一步不是写 `all_reduce`，而是画清楚状态、计算、通信和硬件拓扑。只要这个图清楚，DDP、FSDP、tensor parallel、pipeline parallel 都是不同维度上的切分选择。

11 拓展阅读

- [Stanford CS336 Spring 2026 course page](#)
- [PyTorch distributed documentation](#)
- [NVIDIA NCCL user guide](#)
- [NCCL tests performance notes](#)
- [Sample all-reduce benchmark code](#)
- [Levanter: JAX-based distributed training stack from CRFM](#)

12 总结与延伸

Lecture 7 把并行训练拆回了最基础的通信和切分维度。Collectives 提供分布式程序的通信词汇；硬件互连决定哪些通信可以高频发生；NCCL/PyTorch distributed 把这些原语落到可运行代码；data/tensor/pipeline parallelism 则分别沿 batch、width、depth 切分训练工作。

最终 takeaways

1. 多 GPU 的两个动机是容量和速度：放不下，或想更快。
2. Collective operations 是分布式训练的基础语言。
3. all-reduce = reduce-scatter + all-gather 是理解 DDP 到 FSDP/ZeRO 的关键。
4. Tensor parallel 通信频繁，通常要求 NVLink/NVSwitch；pipeline parallel 通信较粗粒度，但要处理 bubble。
5. 真实训练系统是在 recompute、memory 和 communication 之间换瓶颈。
6. 分布式优化必须用 benchmark/profile 验证，不能只按硬件规格表推断。

和下一讲的连接

本讲先搭起 collectives 和三种基本 parallelism。下一步自然是更系统地处理模型状态分片：ZeRO/FSDP 如何用 all-gather 和 reduce-scatter 降低 replicated state，以及怎样通过 overlap/prefetch 把通信藏到计算后面。